



GOOGLE CLOUD PRODUCTION HANDBOOK

BUILD • SECURE • DEPLOY • OPERATE • SCALE



IAM & SECURITY



NETWORKING



COMPUTE



DATA & STORAGE



KUBERNETES



SERVERLESS



CI/CD & AUTOMATION



OBSERVABILITY



RELIABILITY



COST OPTIMIZATION

- ✓ Production Best Practices
- ✓ Real-World Solutions
- ✓ Troubleshooting Guides
- ✓ Security & Compliance
- ✓ High Availability
- ✓ Cost Optimization

A PRACTICAL GUIDE
FOR ENGINEERS BUILDING
AND OPERATING RELIABLE,
SECURE, AND SCALABLE
WORKLOADS ON
GOOGLE CLOUD.

PRODUCTION-FOCUSED
HANDS-ON
REAL-WORLD
ENTERPRISE READY

VERIQTA
MEDIA



VERIQTA
ACADEMY



VERIQTA
COMMUNITY

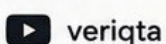


VERIQTA
HIRING NETWORK



20
PAGES

ENGINEER BETTER | DEPLOY FASTER | OPERATE SMARTER | SCALE CONFIDENTLY



veriqta



veriqta



veriqta



veriqta



veriqta

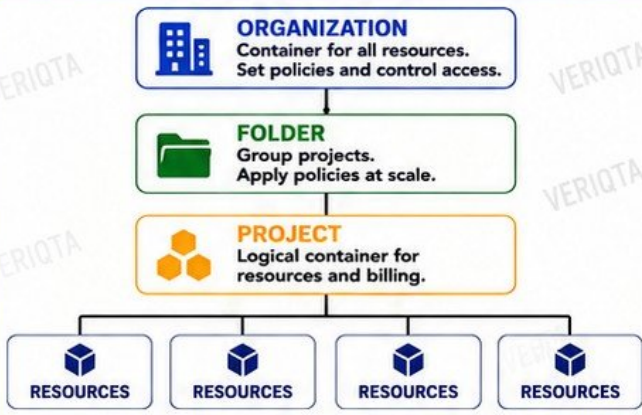


veriqta

1. GOOGLE CLOUD PRODUCTION FOUNDATIONS



1. GOOGLE CLOUD RESOURCE HIERARCHY



2. ORGANIZATIONS, FOLDERS, PROJECTS

- ✓ **ORGANIZATIONS:** Top-level container. Manage access, billing, and policies.
- ✓ **FOLDERS:** Organize projects into logical groups. Apply IAM and org policies.
- ✓ **PROJECTS:** Isolated environments for resources. Each project has its own settings, users, and billing.

BEST PRACTICE



Use folders to separate environments and business units. Apply policies high in the hierarchy.

3. REGIONS AND ZONES

REGION



- ✓ A specific geographic location.
- ✓ Contains one or more zones.
- ✓ Used by most Google Cloud services.

ZONE



- ✓ A data center within a region.
- ✓ Isolated from other zones.
- ✓ Use multiple zones for high availability.

EXAMPLE

Region: us-central1

Zones: us-central1-a, us-central1-b, us-central1-c

4. GLOBAL VS REGIONAL RESOURCES

GLOBAL RESOURCES



- ✓ Available worldwide.
- ✓ Single endpoint.
- ✓ Examples: IAM, Cloud DNS, Cloud CDN, Cloud Storage (multi-region), VPC Global Firewall.

REGIONAL RESOURCES



- ✓ Deployed in a specific region.
- ✓ Used for low latency and data residency.
- ✓ Examples: Compute Engine, Cloud SQL, GKE, Memorystore.

5. PRODUCTION ENVIRONMENT SEPARATION



PRODUCTION
Live workloads.
High availability.



STAGING
Pre-production testing.
Test changes.



DEVELOPMENT
Build and test features.
Early development.



SANDBOX
Experimentation and labs.
Isolated environment.



BEST PRACTICE

Use separate projects for each environment.
Never mix workloads.
Apply least privilege access.

6. SHARED RESPONSIBILITY

GOOGLE'S RESPONSIBILITY



- ✓ Global infrastructure
- ✓ Regions, zones, and network
- ✓ Hardware and physical security
- ✓ Software and services
- ✓ Managed services availability
- ✓ Compliance of the platform

YOUR RESPONSIBILITY



- ✓ Data and access
- ✓ IAM and service accounts
- ✓ Applications and workloads
- ✓ Configuration and updates
- ✓ Network security configuration
- ✓ Monitoring and logging
- ✓ Compliance in your workloads

7. PRODUCTION ARCHITECTURE PRINCIPLES



RELIABILITY
Design for high availability and fault tolerance.



SECURITY
Secure by default.
Apply least privilege and defense in depth.



PERFORMANCE
Optimize for low latency and high throughput.



COST OPTIMIZATION
Right-size resources.
Monitor and optimize costs.



SCALABILITY
Design to scale horizontally and handle growth.



OPERATIONAL EXCELLENCE

Automate, monitor, and improve continuously.

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

2. PRODUCTION IAM AND ACCESS CONTROL



Secure access. Right people. Right permissions. Right resources.

1. PRINCIPALS AND ROLES



PRINCIPALS

Identities that can be granted access.

- ✓ Users
- ✓ Groups
- ✓ Service Accounts
- ✓ Domains
- ✓ Workload Identities



ROLES

Define what actions principals can perform.
Roles contain permissions.

2. ROLE TYPES



BASIC ROLES

Simple roles with a small, fixed set of permissions.
Example: Owner, Editor, Viewer



PREDEFINED ROLES

Google-managed roles for common services.
Example: roles/storage.admin, roles/compute.viewer



CUSTOM ROLES

Create roles with only the permissions you need.
Example: Custom Storage Auditor

3. LEAST PRIVILEGE



GRANT ONLY WHAT IS NECESSARY.

- ✓ Give minimum permissions required to perform the job.
- ✓ Avoid broad roles like Owner.
- ✓ Use predefined or custom roles instead of basic roles.
- ✓ Review permissions regularly.
- ✓ Remove unused access.



BEST PRACTICE

Least privilege reduces risk and blast radius.

4. SERVICE ACCOUNTS



- ✓ Identities for applications and services.
- ✓ Not tied to any individual.
- ✓ Used by VMs, GKE, Cloud Run, and other workloads.
- ✓ Grant roles to service accounts, not users.
- ✓ Enable auditing and accountability.

5. SERVICE ACCOUNT IMPERSONATION



- ✓ Act as a service account without using its key.
- ✓ Uses IAM permission: iam.serviceAccounts.actAs
- ✓ Improves security by avoiding long-lived keys.
- ✓ Supported in gcloud, SDKs, and many Google services.

6. IAM POLICIES



- ✓ Policies bind principals, roles, and resources.
- ✓ Evaluated as ALLOW by default.
- ✓ Use conditions for fine-grained access.
- ✓ Policies can be set at organization, folder, project, or resource level.

POLICY BINDING STRUCTURE

PRINCIPAL + ROLE + RESOURCE + CONDITION
WHO WHAT WHERE WHEN / HOW

7. WORKLOAD IDENTITY FEDERATION



- ✓ Access Google Cloud without service account keys.
- ✓ Federate external identities (AWS, Azure, OIDC, SAML).
- ✓ More secure than key-based authentication.
- ✓ Use for CI/CD, external workloads, and partners.



KEY BENEFIT

Short-lived credentials.
No keys to steal or rotate.

8. PRODUCTION ACCESS PATTERNS



HUMAN ACCESS

Use groups and least privilege.
Avoid direct user access.



WORKLOAD ACCESS

Use service accounts with restricted roles.



TEMPORARY ACCESS

Use IAM conditions and time-bound access.



ADMIN ACCESS

Use break-glass accounts with strong monitoring.



AUDIT EVERYTHING

Enable Cloud Audit Logs and review regularly.



PRODUCTION IAM CHECKLIST

- ✓ Use least privilege for all access
- ✓ Prefer predefined or custom roles
- ✓ Avoid service account keys
- ✓ Use service account impersonation
- ✓ Apply IAM conditions where possible
- ✓ Enable audit logging
- ✓ Use Workload Identity Federation
- ✓ Review and clean up permissions
- ✓ Regularly test and validate access

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

3. PRODUCTION NETWORKING FOUNDATIONS

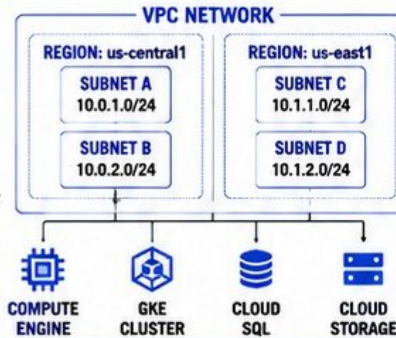


Design secure, reliable, and scalable networks on Google Cloud.

1. VPC ARCHITECTURE



- ✓ VPC is your global, private network in Google Cloud.
- ✓ Isolated from other VPCs by default.
- ✓ Connect resources securely across regions.
- ✓ One VPC per project (auto mode).



2. SUBNETS



- ✓ Divide your VPC into subnets.
- ✓ Each subnet has a CIDR range.
- ✓ Resources are created inside subnets.
- ✓ Subnet types:
 - Regional (recommended)
 - Global (legacy)

SUBNET TYPES



REGIONAL SUBNET
Lives in one region.
High availability.
Can span multiple zones.



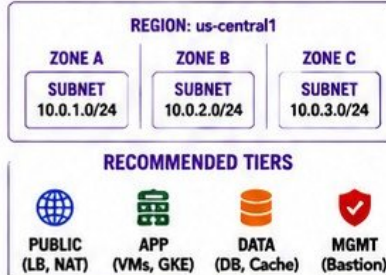
GLOBAL SUBNET
Spans all regions.
Not recommended for most workloads.

3. REGIONAL SUBNET DESIGN



- ✓ Create subnets in all zones of a region.
- ✓ Use /20 or /24 CIDR for most workloads.
- ✓ Separate subnets by workload tier.
- ✓ Design for high availability and scalability.

EXAMPLE: REGIONAL DESIGN



4. ROUTES



- ✓ Routes define where network traffic goes.
- ✓ Each subnet has an implicit local route.
- ✓ Customize routes for hybrid and private access.
- ✓ Route types:
 - Static routes
 - Dynamic routes (BGP)
 - Default routes (0.0.0.0/0)

ROUTE EXAMPLE

DESTINATION	NEXT HOP
10.0.0.0/16	Local
0.0.0.0/0	Internet Gateway
10.10.0.0/16	Cloud VPN
192.168.0.0/16	Cloud Interconnect



BEST PRACTICE
Use the most specific route possible.

5. FIREWALL RULES AND POLICIES



- ✓ Firewall rules control traffic at subnet level.
- ✓ Stateful: return traffic allowed automatically.
- ✓ Use firewall policies for centralized management.
- ✓ Apply least privilege: allow only what is needed.

FIREWALL RULE EXAMPLE

NAME	allow-web
DIRECTION	Ingress
ACTION	Allow
TARGET	Tags: web-server
SOURCE	0.0.0.0/0
PROTOCOL / PORTS	TCP: 80, 443

FIREWALL POLICIES

Hierarchical, reusable, and enforce consistent rules across projects.

TIP

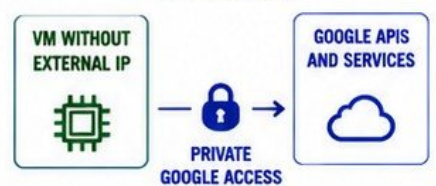
Deny all by default, allow by exception.

6. PRIVATE GOOGLE ACCESS



- ✓ Allows resources without external IPs to access Google APIs and services.
- ✓ Enable on your subnet.
- ✓ Keeps traffic on Google's private network.
- ✓ Required for private workloads to use Google services securely.

HOW IT WORKS



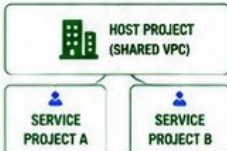
Traffic stays on Google's network.

7. SHARED VPC



- ✓ Centralize network in a host project.
- ✓ Service projects use the network.
- ✓ Simplifies management and security.
- ✓ Enforces network policies at scale.
- ✓ Use IAM for access control.

ARCHITECTURE



8. NETWORK SEGMENTATION



- ✓ Segment networks by tiers and trust levels.
- ✓ Use separate subnets and firewall rules.
- ✓ Reduce blast radius.
- ✓ Improve security and compliance.

EXAMPLE SEGMENTATION

PUBLIC TIER (LB, NAT)	DATA TIER (DB, Cache)
APP TIER (VMs, GKE)	MGMT TIER (Bastion, Admin)

ISOLATE • CONTROL • PROTECT

PRODUCTION NETWORKING CHECKLIST



- ✓ Design VPC and subnets across all regions and zones
- ✓ Use least privilege firewall rules
- ✓ Enable Private Google Access for private workloads
- ✓ Define custom routes for hybrid and private connectivity
- ✓ Use Shared VPC for multi-project environments
- ✓ Segment networks by tiers and trust levels
- ✓ Monitor flow logs and network metrics
- ✓ Review and test network security

4. CLOUD DNS, NAT, AND PRIVATE CONNECTIVITY



Connect resources securely. Reach the world privately. Control name resolution.

1. CLOUD DNS



- ✓ Highly available and globally distributed DNS.
- ✓ Managed authoritative DNS service.
- ✓ Integrates with Load Balancing, Cloud Run, GKE, and more.

COMMON RECORD TYPES

A	- IPv4 address
AAAA	- IPv6 address
CNAME	- Alias record
MX	- Mail exchange
TXT	- Text record
NS	- Name server
SRV	- Service record

2. PUBLIC AND PRIVATE ZONES



PUBLIC ZONE

- ✓ Visible on the internet.
- ✓ Resolves public domains.
- ✓ Used for websites, APIs, mail, and services exposed publicly.
- ✓ Example:

example.com



PRIVATE ZONE

- ✓ Visible only inside VPC.
- ✓ Used for internal name resolution.
- ✓ Associate with VPCs or networks.
- ✓ Example:

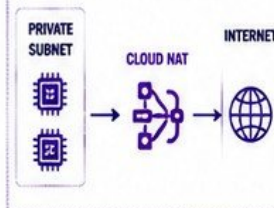
internal.example.local

3. CLOUD NAT



- ✓ Allows resources in private subnets to access the internet securely.
- ✓ No external IPs on VM instances.
- ✓ Supports outbound connections (not for incoming traffic).
- ✓ Provides automatic scaling and high availability.
- ✓ Use with Private Google Access for best results.

HOW CLOUD NAT WORKS

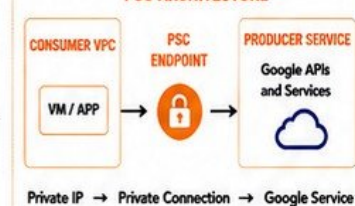


4. PRIVATE SERVICE CONNECT (PSC)



- ✓ Private connectivity to Google APIs and services.
- ✓ Access services using private IPs from your VPC.
- ✓ Keeps traffic on Google's backbone network.
- ✓ Improves security and reduces egress cost.

PSC ARCHITECTURE

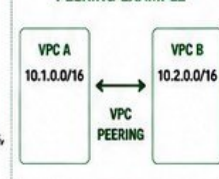


5. VPC NETWORK PEERING



- ✓ Connect two VPC networks privately.
- ✓ Internal IP ranges are reachable.
- ✓ No transitive peering.
- ✓ Supports full mesh or hub-and-spoke topologies.
- ✓ Use cases: shared services, microservices, multi-team isolation.

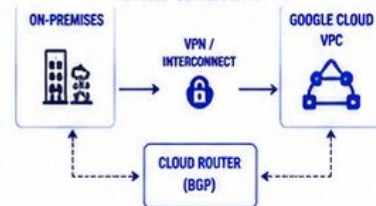
PEERING EXAMPLE



6. HYBRID CONNECTIVITY CONCEPTS

- ✓ Connect on-premises networks to Google Cloud.
- ✓ Options:
 - Cloud VPN (IPsec)
 - Cloud Interconnect (Dedicated)
 - Partner Interconnect
 - HA VPN + Cloud Router
- ✓ Use Cloud Router for dynamic routing (BGP).

HYBRID CONNECTIVITY



7. DNS TROUBLESHOOTING



- ✓ Use dig or nslookup to test.
- ✓ Check record type and value.
- ✓ Verify TTL and caching.
- ✓ Ensure DNS propagation.
- ✓ Check name server delegation.
- ✓ Validate resolver configuration.
- ✓ Check firewall and network connectivity.

USEFUL COMMANDS

```

dig example.com
dig @8.8.8.8 example.com
nslookup example.com
dig +trace example.com
dig -x <IP_ADDRESS>
dig example.com MX
  
```

8. PRODUCTION CONNECTIVITY PATTERNS

PRIVATE WORKLOAD INTERNET ACCESS



Private Subnet
+ Cloud NAT
+ Private Google Access

PRIVATE ACCESS TO GOOGLE APIS



Private Subnet
+ Private Service Connect

MULTI-VPC ARCHITECTURE



VPC Peering
or Shared VPC

HYBRID CONNECTIVITY



Cloud VPN /
Interconnect
+ Cloud Router

INTERNET FACING SERVICES



Load Balancer
+ Public IP
+ Cloud DNS

PRODUCTION NETWORKING CHECKLIST

- ✓ VPCs and subnets designed for scale and isolation
- ✓ Firewall rules follow least privilege
- ✓ Use Private Google Access for private subnets
- ✓ Use Cloud NAT for outbound internet access
- ✓ Use Private Service Connect for Google APIs
- ✓ Peer VPCs or use Shared VPC for connectivity
- ✓ Implement hybrid connectivity with HA
- ✓ Monitor connectivity and DNS health
- ✓ Document IP ranges, routes, and DNS records
- ✓ Test failover and recovery regularly



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta

VERIQTA SOCIAL



X
veriqta



Instagram
veriqta



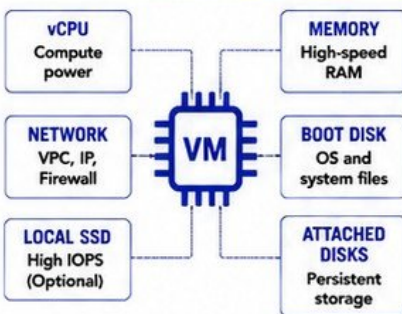
Telegram
veriqta

5. COMPUTE ENGINE IN PRODUCTION



Secure. Reliable. Scalable. Optimized for Production.

1. VM ARCHITECTURE



Highly available, flexible, and fully managed infrastructure.

2. MACHINE FAMILIES

- GENERAL PURPOSE (E2, N2, N2D)**
Balanced compute, memory, networking.
- COMPUTE OPTIMIZED (C3, C3D)**
High performance for compute-intensive workloads.
- MEMORY OPTIMIZED (M3)**
High memory-to-vCPU ratio.
- STORAGE OPTIMIZED (D3)**
High local SSD capacity and throughput.
- ACCELERATOR OPTIMIZED (A2, G2)**
GPUs / TPUs for ML, AI, and HPC.

Choose the right machine for the right workload.

3. IMAGES AND INSTANCE TEMPLATES

IMAGES

- Use Google-provided, custom, or Marketplace images.
- Keep images updated and hardened.
- Use image families for auto-updates.

INSTANCE TEMPLATES

- Define VM configuration once.
- Used by MIGs and autoscalers.
- Ensure consistency across environments.



BEST PRACTICE

Use templates + image families for scalable and consistent deployments.

4. PERSISTENT DISKS

- Standard Persistent Disk (HDD)** – Cost effective.
- Balanced Persistent Disk (SSD)** – Balanced performance and cost.
- SSD Persistent Disk** – High performance and low latency.
- Choose the right size and type.
- Use separate disks for OS, data, and logs.
- Enable disk encryption (default).
- Use snapshots and backups for protection.



BEST PRACTICE

Keep OS disk small and separate data disks for performance and reliability.

5. STARTUP SCRIPTS

- Run automatically when VM starts.
- Use for configuration, package install, and environment setup.
- Supports Bash and PowerShell.
- Store scripts in Cloud Storage and reference in metadata.
- Keep scripts idempotent and fast.

EXAMPLE (BASH)

```
#!/bin/bash
apt-get update -y
apt-get install -y google-cloud-ops-agent
systemctl enable google-cloud-ops-agent
systemctl start google-cloud-ops-agent
```

6. SHIELDED VMS



- Protects VM from boot to shutdown.
- Features:
 - Secure Boot
 - vTPM
 - Integrity Monitoring
 - Confidential VM (optional)
- Prevents rootkits and firmware-level attacks.



BEST PRACTICE

Enable Shielded VM by default for all production workloads.

7. METADATA

- Key-value store attached to VM.
- Used to pass configuration and startup data.
- Accessed securely from within the VM.
- Use for startup scripts, SSH keys, and environment settings.



BEST PRACTICE

Prefer metadata over hardcoding values in scripts or images.

8. PRODUCTION VM CONFIGURATION

- Use custom VPC and private subnets.
- Assign static internal IPs for critical VMs.
- Enable OS Login with IAM for SSH/RDP.
- Apply least-privilege service accounts.
- Enable monitoring and logging agents.
- Use managed instance groups for scalability.
- Regular backups and snapshots.
- Use labels for environment, owner, and cost.



BEST PRACTICE

Build for scalability, security, observability, and cost optimization.

QUICK VM CONFIG CHECKLIST



- Right machine type selected
- Image up to date
- Shielded VM enabled
- OS Login enabled
- Least-privilege service account
- Boot disk encrypted
- Monitoring & logging enabled
- Backups and snapshots configured
- Labels applied
- Firewall rules allow only required traffic

PRODUCTION VM LIFECYCLE



CREATE TEMPLATE

Define VM configuration in an instance template.



LAUNCH VM / MIG

Deploy VM or use MIG for high availability.



SECURE & CONFIGURE

Apply IAM, Shielded VM, metadata, and scripts.



OPERATE & MONITOR

Monitor, patch, and optimize performance.



BACKUP & PROTECT

Snapshot, backup, and ensure recovery.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

VERIQTA SOCIAL

6. MANAGED INSTANCE GROUPS AND AUTOSCALING



Keep your applications available, resilient, and scalable.

1. MANAGED INSTANCE GROUPS (MIGS)



- ✓ MIGs manage identical VM instances.
- ✓ Ensure high availability and scalability.
- ✓ Automatically replace unhealthy VMs.
- ✓ Support autoscaling and rolling updates.
- ✓ Use with Load Balancing for production apps.

MIG ARCHITECTURE



2. INSTANCE TEMPLATES



- ✓ Blueprint for VM configuration.
- ✓ Includes machine type, disks, tags, service accounts, metadata, and more.
- ✓ Immutable – changes require a new template.
- ✓ Used by MIGs to create and replace instances.

TEMPLATE COMPONENTS



MACHINE TYPE



BOOT DISK IMAGE



SERVICE ACCOUNT



NETWORK TAGS



METADATA & STARTUP SCRIPT

3. AUTOSCALING



- ✓ Automatically adjusts the number of instances.
- ✓ Scale based on CPU, custom metrics, or load.
- ✓ Set minimum, maximum, and target values.
- ✓ Improves performance and controls cost.

AUTOSCALING FLOW



4. AUTOHEALING



- ✓ Continuously monitors instance health.
- ✓ Unhealthy instances are automatically replaced.
- ✓ Uses Health Checks to determine health.
- ✓ Ensures your apps stay available.

AUTOHEALING FLOW

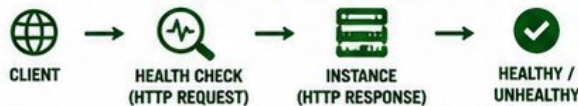


5. HEALTH CHECKS



- ✓ Determine if instances are healthy.
- ✓ Types: HTTP(S), TCP, and gRPC.
- ✓ Configured with check interval and timeout.
- ✓ Used by Autohealing and Load Balancers.

EXAMPLE: HTTP HEALTH CHECK



6. ROLLING UPDATES



- ✓ Update instances without downtime.
- ✓ Replace instances in batches.
- ✓ Choose: Max surge, Max unavailable, or both.
- ✓ Works with Instance Templates.

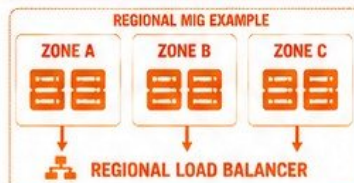
ROLLING UPDATE FLOW



7. REGIONAL MIGS



- ✓ Distribute instances across multiple zones in a region.
- ✓ Provides high availability for zonal failures.
- ✓ Used with Regional Load Balancing.
- ✓ Recommended for production workloads.



8. FAILURE RECOVERY



- ✓ Zonal Failure: Regional MIG automatically recreates instances in healthy zones.
- ✓ Instance Failure: Autohealing replaces unhealthy instances.
- ✓ Scaling Failure: Autoscaler maintains desired capacity.
- ✓ Disaster Recovery: Use backups and global load balancing for region-level recovery.

PRODUCTION MIG CHECKLIST

- ✓ Use Instance Templates
- ✓ Enable Autohealing
- ✓ Configure Health Checks
- ✓ Set Autoscaling Policies
- ✓ Use Rolling Updates
- ✓ Distribute across zones (Regional MIG)
- ✓ Use Load Balancing
- ✓ Monitor and Alert
- ✓ Test Failure Scenarios
- ✓ Regularly Review Settings



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

7. PRODUCTION LOAD BALANCING

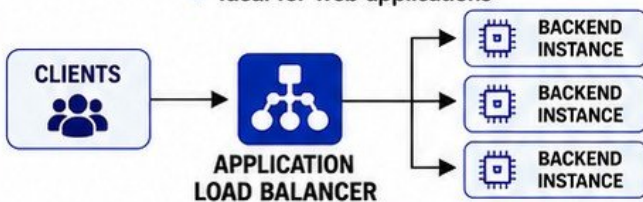


RELIABLE • SCALABLE • SECURE • HIGH PERFORMANCE

1. APPLICATION LOAD BALANCERS (ALB)



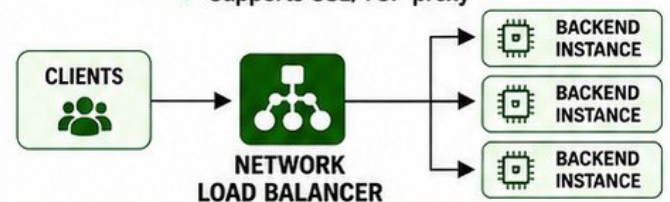
- Layer 7 (HTTP/HTTPS) load balancing
- Advanced traffic routing
- URL maps and host/path rules
- Global or regional
- Ideal for web applications



2. NETWORK LOAD BALANCERS (NLB)



- Layer 4 (TCP/UDP) load balancing
- High performance and low latency
- Global or regional
- Ideal for TCP/UDP workloads
- Supports SSL/TCP proxy

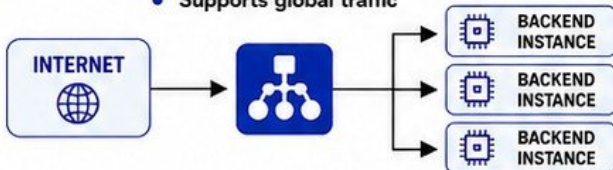


3. INTERNAL VS EXTERNAL LOAD BALANCING

EXTERNAL LOAD BALANCER



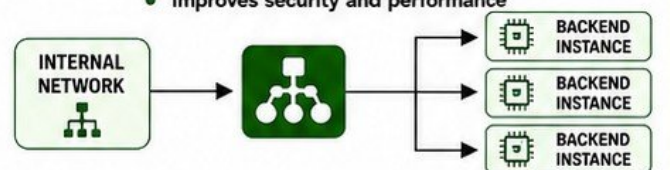
- Accessible from the internet
- Has external IP addresses
- Used for public applications
- Supports global traffic



INTERNAL LOAD BALANCER



- Accessible only within VPC
- Uses internal IP addresses
- Used for internal applications
- Improves security and performance



4. BACKEND SERVICES



- Group of backend resources
- Attach to a load balancer
- Supports MIG, NEG, GCE, GKE
- Health check integration
- Session affinity support

5. HEALTH CHECKS



- Monitors backend health
- HTTP, HTTPS, TCP, SSL checks
- Customizable check intervals
- Unhealthy backends removed
- Improves reliability

6. URL MAPS



- Defines how traffic is routed
- Host rules and path rules
- Routes to backend services
- Supports redirects
- Used by ALB (Layer 7)

7. TLS CERTIFICATES



- Encrypts traffic in transit
- Use managed or self-managed
- Support for SSL services
- Automated certificate manager
- Required for HTTPS

8. PRODUCTION TRAFFIC ROUTING



- Host and path based routing
- Traffic splitting (canary, blue/green)
- Geo-based routing
- Weighted routing
- Fault tolerance and failover

9. PRODUCTION BEST PRACTICES



- Use backend health checks
- Enable HTTPS with TLS
- Design for high availability
- Use appropriate load balancer
- Monitor and optimize traffic

LOAD BALANCER COMPARISON

FEATURE	APPLICATION LOAD BALANCER	NETWORK LOAD BALANCER	INTERNAL LOAD BALANCER	EXTERNAL LOAD BALANCER
LAYER	Layer 7 (HTTP/HTTPS)	Layer 4 (TCP/UDP)	Layer 4 or 7 (Internal)	Layer 4 or 7 (External)
PROTOCOLS	HTTP, HTTPS	TCP, UDP, SSL	TCP, UDP, HTTP, HTTPS	TCP, UDP, HTTP, HTTPS
IP ADDRESS	External	External	Internal	External
USE CASE	Web apps, APIs	High performance, TCP/UDP	Internal services	Public applications
URL MAPS	Yes	No	Yes (if Layer 7)	Yes (if Layer 7)



TIP: Choose the right load balancer for your workload.
Combine features for secure, scalable, and highly available applications.



Google Cloud



8. CLOUD STORAGE IN PRODUCTION



DURABLE • SECURE • SCALABLE • COST-EFFICIENT

1. BUCKETS AND OBJECTS



- Buckets are containers for your data.
- Objects are stored inside buckets.
- Each object has a unique name.
- Supports any type and size of data.

2. STORAGE CLASSES



STANDARD

For frequently accessed data



NEARLINE

For data accessed monthly



COLDLINE

For data accessed quarterly



ARCHIVE

For long-term data archiving with retrieval

3. IAM AND ACCESS CONTROL



- Use IAM to control who can access your data.
- Grant roles at the bucket or object level.
- Follow the principle of least privilege.

4. UNIFORM BUCKET-LEVEL ACCESS



- Enable uniform access for simpler permission management.
- Disables object-level ACLs.
- Use IAM policies for all access control.

5. OBJECT VERSIONING



- Keep multiple versions of an object.
- Protect against accidental deletion or overwrite.
- Easily restore previous versions.

6. LIFECYCLE MANAGEMENT



- Automate the movement of objects to cheaper storage classes.
- Set rules to delete old or unused data.
- Optimize cost without manual work.

7. RETENTION POLICIES



- Lock data for a specific period.
- Prevent deletion or modification.
- Meet compliance and legal requirements.
- Works at bucket level.

8. REPLICATION AND RESILIENCE



- Multi-Regional (Dual-Region) for high availability.
- Regional replication for low latency and resilience.
- Automatic protection against zone or region failures.
- Built for 99.999999999% durability.

PRODUCTION BEST PRACTICES



- Use meaningful bucket names and labels.
- Enable uniform access, versioning, and lifecycle policies.
- Encrypt data in transit and at rest.
- Monitor access and storage usage.

CLOUD STORAGE ARCHITECTURE OVERVIEW



STORE IT. SECURE IT. PROTECT IT. SCALE IT. THAT'S GOOGLE CLOUD STORAGE.



veriqta



veriqta



veriqta

VERIQTA SOCIAL



veriqta



veriqta



veriqta

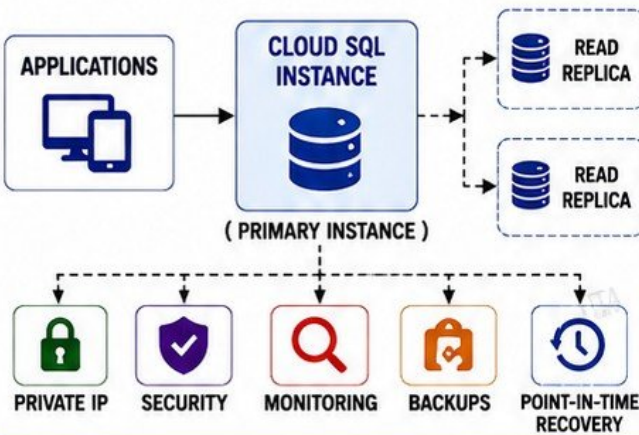


9. CLOUD SQL AND PRODUCTION DATABASES



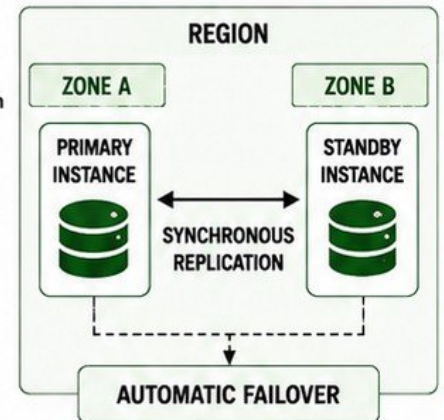
RELIABLE • SECURE • SCALABLE • HIGHLY AVAILABLE

1. CLOUD SQL ARCHITECTURE

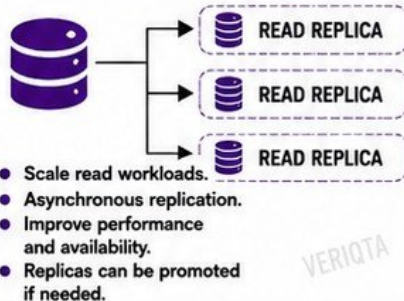


2. HIGH AVAILABILITY

- ✓ Regional HA with automatic failover.
- ✓ Synchronous replication between zones.
- ✓ 99.95% availability SLA.
- ✓ Automatic failover to standby instance.
- ✓ No data loss during failover.



3. READ REPLICAS



- Scale read workloads.
- Asynchronous replication.
- Improve performance and availability.
- Replicas can be promoted if needed.

4. BACKUPS



- Automated daily backups.
- User-defined backup retention.
- Backups stored in Google Cloud Storage.
- Cross-region backup support.

5. POINT-IN-TIME RECOVERY



- Restore your database to any second.
- Uses transaction logs.
- Helps recover from human error or bad data.
- Works with automated backups.

6. PRIVATE IP



- Use private IP for secure connectivity.
- Access only from within your VPC.
- Reduces exposure to the public internet.
- Works with VPC peering and Serverless VPC Access.

7. CONNECTION MANAGEMENT



- Use connection pooling.
- Set max connections appropriately.
- Use Cloud SQL Proxy for secure connections.
- Avoid connection exhaustion.

8. DATABASE MONITORING AND FAILOVER



- Monitor CPU, memory, disk, and connections.
- Set up alerts and dashboards.
- Review slow queries and performance.
- Plan and test failover regularly.

CLOUD SQL PRODUCTION BEST PRACTICES



Use least privilege IAM access.



Encrypt data at rest and in transit.



Test backups and recovery regularly.



Right-size instances for performance and cost.



Keep database engine and maintenance up to date.


BUILD RELIABLE, SECURE, AND HIGH-PERFORMANCE DATABASES ON GOOGLE CLOUD.



10. SERVERLESS PRODUCTION ARCHITECTURE



CLOUD RUN – BUILD. DEPLOY. SCALE. WITHOUT MANAGING SERVERS.

1. CLOUD RUN OVERVIEW



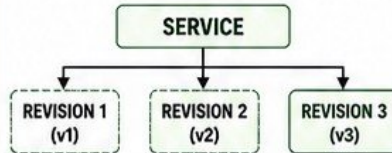
- Fully managed serverless platform for containers.
- Pay only for what you use.
- Scale to zero and beyond.
- Supports HTTP and gRPC.
- Built for production workloads.



2. SERVICES AND REVISIONS



- A service routes traffic to revisions.
- Each deployment creates a new revision.
- Revisions are immutable and versioned.
- Easily roll back to any previous revision.



3. AUTOSCALING



- Automatically scales from zero to thousands of requests.
- Scales up with traffic.
- Scales down to zero when idle.
- No capacity planning required.



4. CONCURRENCY



- Controls how many requests each instance can handle at the same time.
- Higher concurrency = better utilization.
- Tune based on your application type.
- Default: 80 concurrent requests per instance.



5. TRAFFIC SPLITTING



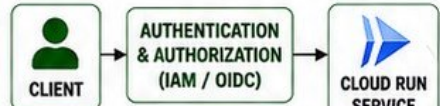
- Split traffic between revisions by percentage.
- Safely test new versions in production.
- Gradually shift traffic to the new revision.
- Instant rollback if needed.



6. AUTHENTICATION



- Integrated with Google Identity Platform.
- Protect services with IAM permissions.
- Require authentication for users and services.
- Easy integration with OIDC and IAM.



7. SERVERLESS VPC ACCESS



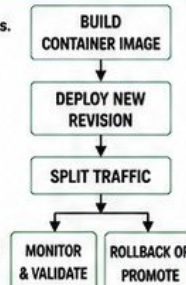
- Securely connect to resources in your VPC.
- Access databases, Redis, Memorystore, and internal services.
- Uses Serverless VPC Access connector.
- No public IP required.



8. PRODUCTION DEPLOYMENT PATTERNS



- Blue/Green Deployments.
- Canary Releases with traffic splitting.
- Automated CI/CD deployments.
- Health checks and automatic rollbacks.
- Infrastructure as Code (Terraform, gcloud, etc.)



CLOUD RUN BENEFITS

- ✓ ZERO SERVER MANAGEMENT
- ✓ PAY-PER-REQUEST MODEL
- ✓ AUTOMATIC SCALING
- ✓ HIGH AVAILABILITY
- ✓ FAST DEPLOYMENTS
- ✓ BUILT FOR MODERN APPS
- ✓ FULLY MANAGED PLATFORM



FOCUS ON YOUR CODE, NOT YOUR INFRASTRUCTURE.
CLOUD RUN POWERS MODERN, SCALABLE APPLICATIONS.



Google Cloud



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA SOCIAL

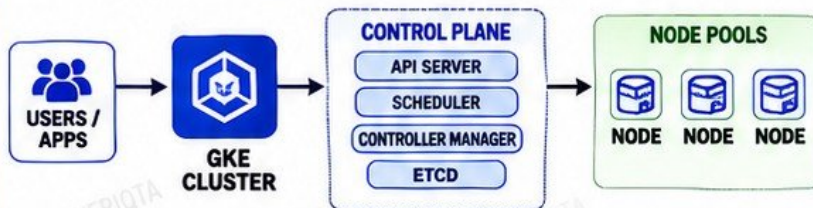


11. GOOGLE KUBERNETES ENGINE IN PRODUCTION



ORCHESTRATE • SCALE • SECURE • RUN WORKLOADS RELIABLY

1. GKE ARCHITECTURE



2. STANDARD VS AUTOPILOT

FEATURE	STANDARD	AUTOPILOT
Cluster control	Full control	Managed
Node management	Your choice	Google managed
Ops overhead	Higher	Lower
Cost model	Node + pod	Per workload
Best for	Custom needs	Simplicity

3. REGIONAL CLUSTERS



- Run clusters in multiple regions for high availability.
- Keep workloads close to users.
- Improve disaster recovery.
- Avoid single-region failure.

4. NODE POOLS



- Create separate node pools for different workloads.
- Use system and workload node pools.
- Upgrade node pools without downtime.

5. WORKLOAD IDENTITY FEDERATION FOR GKE



- Use Workload Identity for secure access to Google Cloud APIs.
- Avoid long-lived service account keys.
- Give pods least-privilege access.

6. INGRESS AND SERVICES



- Use Ingress for external HTTP(S) traffic.
- Expose apps with Services.
- Choose ClusterIP, NodePort or LoadBalancer.
- Use Ingress rules for routing.

7. AUTOSCALING



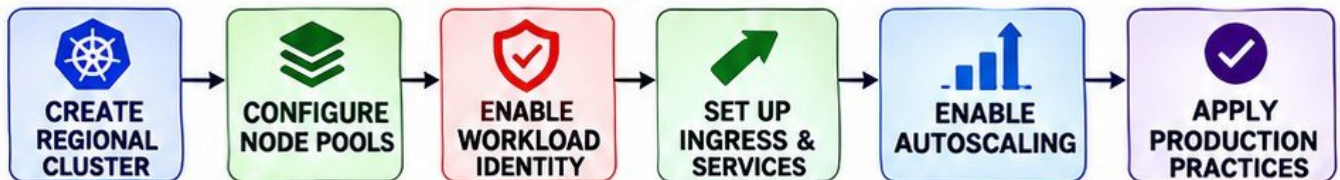
- Use Horizontal Pod Autoscaler (HPA).
- Scale based on CPU, memory or custom metrics.
- Keep workloads responsive.

8. PRODUCTION CLUSTER PRACTICES



- Run regional clusters.
- Use node pools and workload separation.
- Enable monitoring and logging.
- Apply security best practices.
- Keep clusters upgraded.

GKE PRODUCTION FLOW



TIP: RUN GKE IN REGIONAL MODE WITH WORKLOAD IDENTITY AND PROPER AUTOSCALING FOR A SECURE, HIGHLY AVAILABLE PRODUCTION ENVIRONMENT.



veriqta



veriqta



veriqta

VERIQTA SOCIAL



veriqta



veriqta



veriqta

12. MESSAGING AND EVENT-DRIVEN SYSTEMS

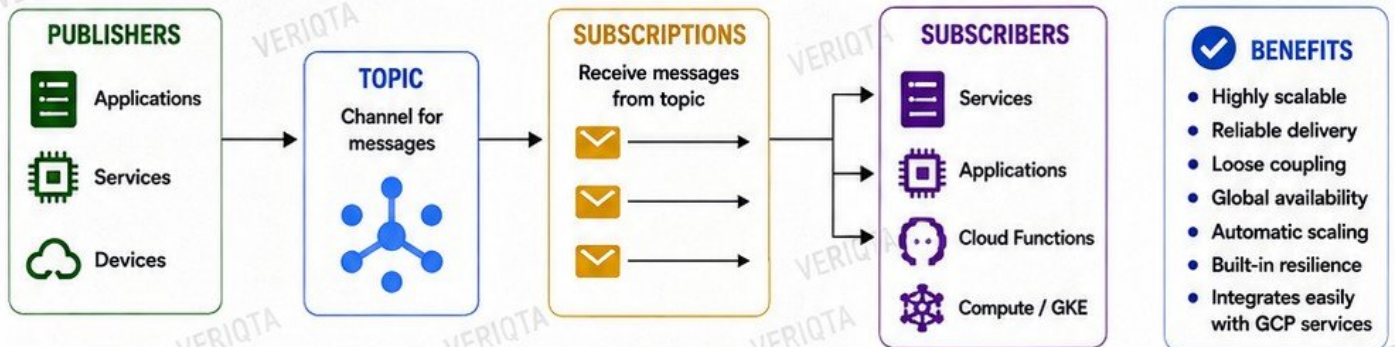


DECOUPLE • ASYNCHRONOUS • SCALABLE • RESILIENT • RELIABLE

CLOUD PUB/SUB OVERVIEW



Google Cloud Pub/Sub is a fully managed messaging service for reliable, scalable, asynchronous, and event-driven applications.



1. TOPICS AND SUBSCRIPTIONS



A topic is a named bus to which publishers send messages.

Subscriptions receive messages from the topic.

One topic can have multiple subscriptions.

2. PUSH VS PULL



PUSH

Messages are pushed to an endpoint (HTTP/S). Best for webhooks and serverless.



PULL

Subscribers pull messages using an API. Best for batch and worker apps.

3. ACKNOWLEDGEMENTS



Messages must be acknowledged after successful processing.



Unacknowledged messages become eligible for redelivery after the ack deadline.



At-least-once delivery is guaranteed.

4. RETRY POLICIES



If a message processing fails, Pub/Sub retries delivery.



Configurable retry backoff and maximum delivery attempts.

Ensures higher reliability.

5. DEAD-LETTER TOPICS



Messages that exceed max delivery attempts are sent to a dead-letter topic.

Helps isolate and inspect poison messages.

Prevent endless retry loops.

6. EVENT-DRIVEN ARCHITECTURE



Services publish events to Pub/Sub.

Other services subscribe and react to events.

Decouples producers and consumers.

Enables scalable microservices architecture.

7. FAILURE HANDLING



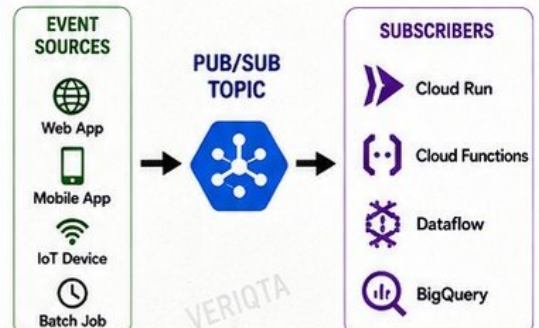
Automatic retries for transient failures.

Dead-letter topics for permanent failures.

Idempotent processing recommended.

Monitor, alert, and recover quickly.

EVENT-DRIVEN ARCHITECTURE EXAMPLE



KEY TAKEAWAYS



Use Pub/Sub for asynchronous, event-driven communication.



Choose push for webhooks and pull for worker applications.



Always acknowledge messages after successful processing.



Use dead-letter topics to handle poison messages cleanly.



Design for failures, retries, and idempotent processing.

VERIQTA SOCIAL



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

13. INFRASTRUCTURE AS CODE AND AUTOMATION



AUTOMATE • REPEATABLE • CONSISTENT • SCALABLE • RELIABLE



TERRAFORM WITH GOOGLE CLOUD

Terraform is an Infrastructure as Code tool that allows you to build, change, and version infrastructure safely and efficiently on Google Cloud.

1. PROVIDER CONFIGURATION



- Configure the Google Cloud provider.
- Authenticate using Service Account.
- Set project, region, and zone.
- Use variables for flexibility.

```
provider "google" {
  project = var.project_id
  region  = var.region
  zone    = var.zone
  credentials = file("key.json")
}
```

2. REMOTE STATE



- Store state remotely in Google Cloud Storage.
- Enables collaboration and state locking.
- Prevents state conflicts and corruption.
- Use a dedicated bucket with versioning.

```
terraform {
  backend "gcs" {
    bucket = "my-tf-state-bucket"
    prefix = "envs/prod"
  }
}
```

3. MODULES



- Reusable, modular infrastructure.
- Encapsulate resources into modules.
- Promote DRY (Don't Repeat Yourself).
- Version and share modules.

```
module "vpc" {
  source = "../modules/vpc"
  project_id = var.project_id
  region   = var.region
}
```

4. ENVIRONMENT SEPARATION



- Separate environments using workspaces or directory structure.
- Use separate state files per environment.
- Isolate resources and credentials.

```
# Using workspace
terraform workspace
new prod
terraform workspace
select prod
```

5. IAM AUTOMATION



- Manage IAM roles and service accounts.
- Enforce least privilege.
- Automate IAM bindings and policies.
- Use IAM modules for consistency.

```
resource "google_project_iam_member"
"editor" {
  project = var.project_id
  role    = "roles/editor"
  member  = "serviceAccount:S{
    google_service_account.app.email}"
}
```

6. INFRASTRUCTURE PIPELINES



- Automate plan, apply, and destroy.
- Integrate with Cloud Build, GitHub Actions, or GitLab CI.
- Enforce policy and validation.
- Require approvals for production.

```
1. terraform fmt -check
2. terraform init
3. terraform validate
4. terraform plan
5. terraform apply
```

7. STATE AND CHANGE MANAGEMENT



- Track infrastructure changes as code.
- Review plans before apply.
- Lock state during operations.
- Backup state regularly.
- Use version control for all code.

```
terraform plan
# Review the plan
terraform apply
```

8. BEST PRACTICES



- Keep modules small and focused.
- Use variables and outputs.
- Enable state locking.
- Tag all resources.
- Follow naming conventions.
- Document your code.
- Automate everything possible.

INFRASTRUCTURE AS CODE WORKFLOW



WRITE CODE
Define resources in Terraform.



COMMIT & PUSH
Push code to version control.



RUN PIPELINE
Validate, plan, and apply.



PROVISION
Infrastructure is created.



MANAGE & MONITOR
Monitor, update, and iterate.



KEY TAKEAWAY

Infrastructure as Code brings automation, consistency, and control to your Google Cloud environments. Code it, version it, test it, and deploy it with confidence.



Google Cloud



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA SOCIAL

14. PRODUCTION CI/CD

AUTOMATE • STANDARDIZE • SECURE • REPEATABLE • RELIABLE



Google Cloud provides a complete CI/CD solution to build, store, and deploy your applications securely and at scale.

1. CLOUD BUILD



- Fully managed CI/CD service
- Build containers and artifacts
- Run tests and security scans
- Trigger on push or PR
- Integrates with GitHub, GitLab, and Bitbucket

steps:

```
- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'IMAGE_URI', '.']
- name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'IMAGE_URI']
```

2. ARTIFACT REGISTRY



- Store and manage container images
- Docker, Maven, npm, Python, Go
- Multi-region and high availability
- IAM-based access control
- Vulnerability scanning integration

```
gcloud artifacts repositories create my-repo \
  --repository-format=docker \
  --location=us-central1 \
  --description="Docker repository"
```

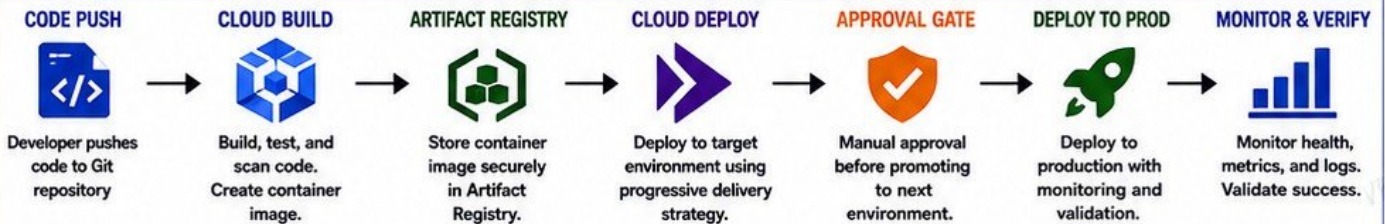
3. CLOUD DEPLOY



- Continuous delivery for Kubernetes
- Progressive delivery strategies
- Blue/Green and Canary deployments
- Automatic rollbacks on failure
- Integrates with Cloud Build and Artifact Registry

```
gcloud deploy releases create release-1
  --delivery-pipeline=my-pipeline
  --images=app=IMAGE_URI
```

4. PRODUCTION CI/CD PIPELINE FLOW



5. BUILD PIPELINES



- Automate build, test, and quality checks
- Use Cloud Build triggers
- Parallel and reliable builds
- Reusable steps with YAML configuration

steps:

```
- name: 'gcr.io/cloud-builders/mvn'
  args: ['clean', 'package']
- name: 'gcr.io/cloud-builders/gcloud'
  args: ['tests', 'run']
```

6. CONTAINER IMAGE MANAGEMENT



- Use Artifact Registry for secure storage
- Scan images for vulnerabilities
- Apply metadata and tags
- Lifecycle and cleanup policies

```
gcloud artifacts docker images list
  --repository=my-repo --location=us-central1
gcloud artifacts docker images delete
  IMAGE_URI --quiet
```

7. DEPLOYMENT PIPELINES



- Define multi-environment pipelines
- Promote artifacts safely
- Progressive delivery (Canary/Blue-Green)
- Automatic validation at each stage

```
gcloud deploy delivery-pipelines create
  my-pipeline --region=us-central1
  --description="Prod pipeline"
```

8. APPROVAL GATES



- Add manual approvals before production
- Enforce business and security policies
- Audit and track approvals

```
gcloud deploy targets update prod
  --require-approval
  --approval-policies=policy.yaml
```

9. ROLLBACK STRATEGIES



- Automatic rollback on failed deployments
- Manual rollback with one command
- Keep previous releases available

```
gcloud deploy releases rollback
  release-1 --delivery-pipeline=my-pipeline
  --to-revision=previous
```

BEST PRACTICES

- Use version control for all code and configurations
- Automate tests, security scans, and quality gates
- Use least privilege IAM for all services
- Use separate projects for dev, staging, and prod
- Monitor deployments and set alerts
- Document and version your pipeline configurations

ROLLBACK STRATEGIES

- Automatic rollback on health check failure
- Rollback to last known good release
- Blue/Green switch back to previous environment
- Canary rollback by adjusting traffic to 0%
- Database migrations with backward compatibility

KEY BENEFITS

- Faster and more reliable releases
- Consistent and repeatable deployments
- Reduced risk with automated rollbacks
- Full visibility and auditability
- Enables DevSecOps at scale



Google Cloud

15. OBSERVABILITY AND MONITORING



SEE EVERYTHING • UNDERSTAND EVERYTHING • IMPROVE EVERYTHING



Observability gives you deep visibility into the health, performance, and reliability of your systems so you can act before users are impacted.

1. CLOUD MONITORING



- Collects and visualizes metrics from Google Cloud and your apps
- Out-of-the-box metrics and custom metrics
- Multi-resource dashboards
- Aggregations and group-by
- SLO and error budget metrics

EXAMPLE: CPU UTILIZATION METRIC

```
metric.type="compute.googleapis.com/instance/cpu/utilization"
resource.type="gce_instance"
alignment.period="60s"
per_series_aligner="ALIGN_MEAN"
```

2. CLOUD LOGGING



- Centralized log collection and analysis
- Query logs using Logs Explorer
- Log-based metrics
- Log sinks to BigQuery, Pub/Sub, or GCS
- Audit logs for security and compliance

EXAMPLE: LOGS EXPLORER QUERY

```
resource.type="gce_instance"
severity>=ERROR
jsonPayload.status="500"
```

3. METRICS



Numerical time-series data about system performance.

Examples:
CPU, memory,
disk, latency,
request count.

4. LOGS



Detailed records of events from applications and infrastructure.

Examples:
Access logs,
error logs,
audit logs.

5. DASHBOARDS



Visualize metrics and logs in one place.

Create custom dashboards for your teams and services.

6. ALERTING POLICIES



Detect issues early and notify the right people.

Set conditions on metrics, logs, or uptime checks.

7. UPTIME CHECKS



Actively check your services from global locations.

Monitor HTTP(S), TCP, and custom endpoints.

8. SLI, SLO AND PRODUCTION VISIBILITY



Define reliability goals and measure what matters.

SLI = Indicator
SLO = Objective
Error Budget =
Reliability currency

9. OBSERVABILITY FLOW



APPLICATIONS & SERVICES

Your apps generate metrics and logs.



METRICS

Collected by Cloud Monitoring.



LOGS

Collected by Cloud Logging.



MONITORING & LOGGING

Analyze, correlate, and visualize.



DASHBOARDS

Understand health and performance.



ALERTS

Get notified on issues.



ACTION

Investigate and resolve fast.

10. COMMON ALERTING CONDITIONS

- ✓ High CPU utilization
- ✓ High memory usage
- ✓ Disk space running low
- ✓ High error rate (5xx)
- ✓ High latency
- ✓ Service down

11. BEST PRACTICES

- ✓ Use labels for better filtering and grouping.
- ✓ Create SLOs for critical user journeys.
- ✓ Alert on symptoms, not on noise.
- ✓ Use dashboards for every service.
- ✓ Review alerts regularly.
- ✓ Instrument your applications.



TIP: You can't improve what you can't see. Build visibility, set goals, and act fast.



Google Cloud



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA SOCIAL

16. GOOGLE CLOUD SECURITY IN PRODUCTION



PROTECT • DETECT • RESPOND • COMPLY • CONTINUOUSLY IMPROVE



Security in Google Cloud is built on a shared responsibility model. These services and practices help you secure data, workloads, and infrastructure in production.

1. SECRET MANAGER



- Store and manage secrets like passwords, API keys, and certificates.
- Automatic versioning and replication.
- Fine-grained IAM access control.

```
# Create a secret
gcloud secrets create db-pass
```

```
# Access a secret version
gcloud secrets versions access latest --secret=db-pass
```

2. CLOUD KMS



- Encrypt and protect your data at rest.
- Manage encryption keys centrally.
- Integrate with services like Cloud Storage, Cloud SQL, and GKE.

```
# Create a key ring
gcloud kms keyrings create prod-keyring --location=global
```

```
# Create a key
gcloud kms keys create app-key --keyring=prod-keyring --location=global
```

3. SECURITY COMMAND CENTER



- Unified view of security posture and risks.
- Threat detection and vulnerability management.
- Asset inventory and security findings.

```
# Enable SCC
gcloud services enable securitycenter.googleapis.com

# View findings
gcloud scc findings list
```

4. ORGANIZATION POLICIES



- Set guardrails and enforce security policies.
- Restrict locations, services, and resource configurations.
- Apply policies at org, folder, or project level.

```
# Disable public IPs on VMs
gcloud resource-manager org-policies set-policy deny-external-ip.yaml --organization=123456789012
```

5. VPC SERVICE CONTROLS



- Prevent data exfiltration from GCP services.
- Define service perimeters around resources.
- Control access from trusted networks.

```
# Create a service perimeter
gcloud access-context-manager perimeters create prod-perimeter --policy=access-policy.yaml
```

6. ENCRYPTION



- Encrypt data in transit (TLS by default).
- Encrypt data at rest using Google-managed or customer-managed keys.
- Customer-managed keys with Cloud KMS for control.

```
# Enable CMEK for a bucket
gsutil kms encryption -k projects/PROJECT/locations/global/keyRings/prod-keyring/cryptoKeys/app-key \ gs://my-secure-bucket
```

7. AUDIT LOGS



- Record API calls and administrative actions.
- Use Admin Activity, Data Access, and System Event logs.
- Send logs to Cloud Logging or BigQuery for analysis.

```
# Enable audit logs
gcloud logging settings set --organization=123456789012 --enable-all
```

8. DEFENSE-IN-DEPTH ARCHITECTURE



- Layer multiple controls across people, process, and technology.
- Combine IAM, networking, encryption, monitoring, and policies.
- Assume breach and minimize blast radius.

DEFENSE-IN-DEPTH SECURITY LAYERS

IDENTITY



IAM roles, least privilege, MFA

+

PERIMETER



VPC Service Controls, firewalls, Cloud Armor

+

WORKLOAD



Shielded VMs, GKE security, hardening

+

DATA



Encryption, KMS, Secret Manager

+

DETECTION



SCC, Logging, Monitoring

+

RESPONSE



Alerts, incident response, IAM



TIP: Security is not a product, it's a process. Continuously monitor, review, and improve your security posture.

VERIQTA SOCIAL



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

17. RELIABILITY, BACKUP, AND DISASTER RECOVERY



BUILD RESILIENT SYSTEMS • PREPARE FOR FAILURE • RECOVER FAST



Design for failure, not just for scale. These practices and patterns help you keep your systems available and recover quickly when disasters happen.

1. HIGH AVAILABILITY



- Design for continuous availability.
- Eliminate single points of failure.
- Use managed services when possible.
- Distribute across zones and regions.

EXAMPLES:
Multi-zone GKE, Regional Cloud SQL, Global Load Balancing, Spanner

2. ZONAL VS REGIONAL FAILURES

ZONAL FAILURE



- Affects one zone in a region.
- Mitigate with multi-zone deployment.

REGIONAL FAILURE



- Affects entire region.
- Mitigate with multi-region architecture.

PLAN FOR BOTH. ASSUME FAILURE WILL HAPPEN.

3. MULTI-REGION ARCHITECTURE



- Deploy across multiple regions.
- Use global load balancing.
- Replicate data across regions.
- Use active-active or active-passive patterns.

EXAMPLES:
GKE Multi-Cluster, Cloud Spanner, BigQuery, Cloud Storage Dual-Region

4. BACKUP STRATEGIES



- Automated and scheduled backups.
- Use managed backups where possible.
- Store backups in separate region or project.
- Test restore regularly.

EXAMPLES:
Cloud SQL backups, GCS versioning, Persistent Disk snapshots

5. RTO AND RPO



RTO (RECOVERY TIME OBJECTIVE)

The maximum acceptable time to restore a system.

RPO (RECOVERY POINT OBJECTIVE)

The maximum acceptable data loss measured in time.

DEFINE CLEAR RTO AND RPO FOR EVERY CRITICAL SERVICE.

6. DISASTER RECOVERY PATTERNS



ACTIVE-ACTIVE
Both regions are active. Traffic is served from both regions.



ACTIVE-PASSIVE
One region is active. Second region is standby for failover.



BACKUP & RESTORE
Restore workloads in another region when needed.

CHOOSE THE PATTERN THAT MATCHES YOUR RTO, RPO, AND COST.

7. FAILOVER



- Automate failover where possible.
- Use health checks and monitoring to trigger failover.
- Keep DNS TTL low for faster traffic switching.
- Validate application health after failover.

FAILOVER SHOULD BE FAST, AUTOMATED, AND TESTED.

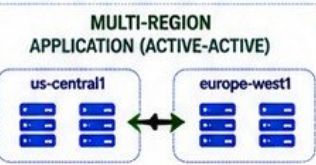
8. RECOVERY TESTING



- Test backups and restores.
- Run DR drills regularly.
- Validate failover and data integrity.
- Document runbooks.
- Continuously improve.

TEST OFTEN. TRUST ONLY WHAT YOU TEST.

RELIABILITY ARCHITECTURE EXAMPLE



DISASTER SCENARIO EXAMPLE



REGION FAILURE
Primary region becomes unavailable



DETECTION
Monitoring detects outage via health checks



FAILOVER TRIGGERED
Traffic is routed to healthy region automatically



SERVICE CONTINUES
Users experience minimal or no downtime



RECOVERY & REVIEW
Root cause analysis, recovery, and improvements



TIP: Plan for the worst, design for resilience, test always, and you will recover with confidence.



Google Cloud

18. COST AND PERFORMANCE ENGINEERING

OPTIMIZE COSTS • IMPROVE PERFORMANCE • DELIVER MORE VALUE



Cost and performance engineering ensures you get the best performance for the lowest possible cost—without compromising reliability or user experience.

1. BILLING ACCOUNTS



- Organize your costs using a billing account hierarchy.
- Separate environments, teams, and projects.
- Use folders and projects to control access and visibility.

View billing accounts
`gcloud billing accounts list`

2. BUDGETS AND ALERTS



- Create budgets to track spend against thresholds.
- Set alerts to notify teams before limits are exceeded.
- Monitor daily, monthly, or quarterly spend.

Create a budget
`gcloud beta billing budgets create \`
`--billing-account=ACCOUNT_ID \`
`--display-name="Monthly Budget"`

3. LABELS AND COST ATTRIBUTION



- Use labels to track costs by team, app, or purpose.
- Ensure consistent label keys across resources.
- Analyze costs using labels in Cost Explorer.

Example label
key: environment
value: production

4. RIGHTSIZING



- Choose the right machine type for your workload.
- Use recommendations from Recommender.
- Remove underutilized resources.

Get rightsizing recommendations
`gcloud recommender recommendations list`

5. AUTOSCALING



- Automatically scale based on demand.
- Use autoscaling for GCE, GKE, and Cloud Run.
- Reduce costs by scaling to zero when possible.

Example: GCE autoscaler
`gcloud compute instance-groups managed create my-group \`
`--enable-autoscaling`

6. COMMITTED USE DISCOUNTS



- Commit to use for 1 or 3 years for significant discounts.
- Apply to Compute Engine, GKE, Cloud SQL, and more.
- Review utilization to maximize savings.

View committed use discounts
`gcloud billing commitments list`

7. STORAGE OPTIMIZATION



- Use the right storage class for your data.
- Move infrequent data to Nearline or Coldline.
- Delete unnecessary snapshots and objects.

Example: Set storage class
`gsutil storageclass set NEARLINE gs://BUCKET/*`

8. PERFORMANCE VS COST DECISIONS



- Balance performance requirements with cost.
- Use caching, CDN, and async processing.
- Continuously test and measure impact.

Always challenge:
"Do we need this level of performance?"

COST OPTIMIZATION CONTINUOUS CYCLE



MEASURE

Track and analyze costs regularly.



IDENTIFY

Find inefficiencies and anomalies.



OPTIMIZE

Apply best practices and rightsize.



AUTOMATE

Use policies, budgets, and autoscaling.



REVIEW

Continuously improve and iterate.

BEST PRACTICES

- ✓ Use budgets and alerts for every environment.
- ✓ Apply labels consistently to all resources.
- ✓ Review costs and recommendations weekly.
- ✓ Automate scaling and shutdown of unused resources.
- ✓ Educate teams on cost ownership and accountability.

KEY METRICS TO TRACK

- Cost per service / per user
- Resource utilization (%)
- Budget vs actual spend
- Committed use discount utilization
- Storage class distribution
- Performance vs cost impact



TIP: The cloud is elastic, but your budget isn't. Optimize continuously. Right resources. Right size. Right cost.



Google Cloud



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



19. REAL PRODUCTION TROUBLESHOOTING

OBSERVE • ANALYZE • ISOLATE • RESOLVE • PREVENT



A PRACTICAL GUIDE TO FIND, ISOLATE, AND RESOLVE PRODUCTION ISSUES FAST AND EFFECTIVELY

1. VM UNREACHABLE



- Check instance status
- Verify firewall rules
- Confirm network tags
- Validate routes
- Check internal IP
- Review OS and services
- Check serial console logs

TOOLS: VPC FLOW LOGS, SERIAL CONSOLE, NETWORK ANALYZER

2. LOAD BALANCER UNHEALTHY BACKENDS



- Check health check results
- Verify backend service config
- Confirm firewall rules
- Verify instance readiness
- Review instance logs
- Check capacity and limits

TOOLS: CLOUD CONSOLE, HEALTH CHECKS, LOGGING, MONITORING

3. IAM PERMISSION DENIED



- Verify principal and identity
- Check roles and permissions
- Review IAM policy bindings
- Check resource hierarchy
- Validate service account access
- Use Policy Troubleshooter

TOOLS: IAM POLICY ANALYZER, POLICY TROUBLESHOOTER, AUDIT LOGS

4. CLOUD RUN DEPLOYMENT FAILURES



- Check build and deploy logs
- Verify service configuration
- Validate environment variables
- Check container startup
- Review IAM permissions
- Check concurrency and limits

TOOLS: CLOUD BUILD LOGS, CLOUD RUN LOGS, ERROR REPORTING

5. GKE WORKLOADS UNAVAILABLE



- Check pod status and events
- Review deployment/replicaset
- Check node health and capacity
- Verify resource requests/limits
- Check liveness and readiness
- Review logs and probes

TOOLS: KUBECTL, GKE CONSOLE, CLOUD LOGGING, MONITORING

6. CLOUD SQL CONNECTION EXHAUSTION



- Check active connections
- Review connection limits
- Configure connection pooling
- Close idle connections
- Tune database parameters
- Enable Cloud SQL Insights

TOOLS: CLOUD SQL INSIGHTS, MONITORING, DATABASE LOGS

7. DNS AND NETWORKING FAILURES



- Check DNS resolution
- Verify DNS records and TTL
- Review VPC firewall rules
- Check subnets and routes
- Validate private access
- Use network diagnostics

TOOLS: CLOUD DNS, VPC FLOW LOGS, NETWORK CONNECTIVITY TEST

8. LOGGING AND INCIDENT INVESTIGATION



- Centralize logs in Cloud Logging
- Use logs-based metrics
- Create dashboards and alerts
- Correlate across services
- Trace requests end-to-end
- Export logs for deep analysis

TOOLS: CLOUD LOGGING, LOG EXPLORER, MONITORING, TRACE, ERROR REPORTING

9. STRUCTURED TROUBLESHOOTING WORKFLOW



1. Define the problem
2. Gather symptoms
3. Check recent changes
4. Isolate the scope
5. Form a hypothesis
6. Test and validate
7. Implement fix
8. Verify and monitor
9. Document and prevent

FOLLOW A CONSISTENT PROCESS TO SOLVE PROBLEMS FASTER AND PREVENT RECURRENCE

QUICK CHECKLIST

- ✓ Always check the basics first: status, health, and connectivity.
- ✓ Use the right tools: logs, metrics, and diagnostic services.
- ✓ Narrow the problem down with a systematic approach.
- ✓ Validate fixes and monitor for stability.
- ✓ Document root cause and update runbooks.



BEST PRACTICES

- ✓ Enable monitoring, logging, and alerting everywhere.
- ✓ Use least privilege and proper IAM roles.
- ✓ Automate health checks and self-healing.
- ✓ Test failure scenarios regularly.
- ✓ Build runbooks for common incidents.



TIP: OBSERVABILITY + PROCESS + PRACTICE = FAST RESOLUTION + RELIABLE SYSTEMS



veriqta



veriqta



veriqta

VERIQTA SOCIAL



veriqta



veriqta



veriqta

20. FINAL PRODUCTION ARCHITECTURE AND REVIEW



DESIGN • BUILD • SECURE • OPERATE • OPTIMIZE

DESIGN A PRODUCTION-GRADE GOOGLE CLOUD ENVIRONMENT

1. PROJECT & ENVIRONMENT STRUCTURE



- Organization
- Folders
- Projects per environment
– Dev / Test / Staging / Prod
- Least-privilege access

2. SHARED VPC

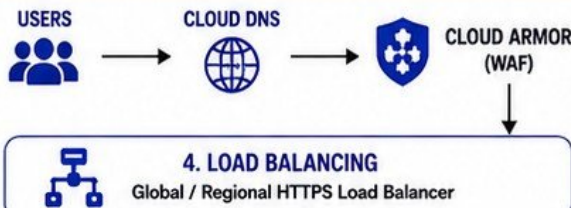


- Centralized network in Host Project
- Service Projects for workloads
- Firewall policies and security controls

3. PUBLIC & PRIVATE WORKLOADS

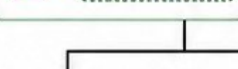


- Public workloads via Load Balancers
- Private workloads in private subnets
- Cloud NAT for outbound internet access



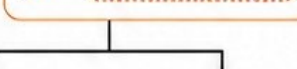
5A. GKE APPLICATION TIER

Private Subnets



5B. CLOUD RUN APPLICATION TIER

Serverless



6. MANAGED DATABASE

Cloud SQL / AlloyDB



7. CLOUD STORAGE

Buckets



8. IAM & SECRETS

IAM, Service Accounts, Secret Manager



9. MONITORING & LOGGING



- Cloud Monitoring
- Cloud Logging
- Alerting Policies
- Dashboards
- Uptime Checks

10. CI/CD PIPELINE



- Cloud Build
- Artifact Registry
- Cloud Deploy
- Multi-stage deployments
- Rollbacks

11. HIGH AVAILABILITY



- Multi-zone / Multi-region
- Regional clusters or services
- Health checks
- Automatic failover

12. BACKUP & DISASTER RECOVERY



- Automated backups
- Cross-region replication for critical data
- RTO / RPO defined
- DR testing and runbooks

13. SECURITY REVIEW



- IAM least privilege
- VPC-SC & firewall rules
- Encryption (at rest and in transit)
- Vulnerability scanning
- Audit logs enabled

14. COST REVIEW



- Budget alerts
- Rightsize resources
- Committed use discounts
- Storage class optimization
- Continuous cost analysis

15. FINAL PRODUCTION-READINESS CHECKLIST



- Architecture reviewed
- Security validated
- Monitoring & alerts active
- Backups tested
- DR plan validated
- Performance tested
- Cost optimized
- Documentation complete
- Go-live approved

PRODUCTION ARCHITECTURE SUMMARY



KEY PRINCIPLES

- ✓ Design for reliability and high availability
- ✓ Secure by default, least privilege access
- ✓ Automate everything (CI/CD + IaC)
- ✓ Monitor everything, alert early
- ✓ Optimize for performance and cost
- ✓ Test failure, plan recovery
- ✓ Document and review continuously

OUTCOME



A secure, scalable, highly available, cost-effective, and well-monitored Google Cloud environment ready for production.



TIP: A well-architected cloud environment is not just about building it, but continuously reviewing, improving, and operating it.



Google Cloud



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA SOCIAL